

Softwarekonzept V2

ClimateCanary – Raumklima Analyse für Firmen

PS Software Engineering SS2026

Maria Kuhn Fabienne Schedler Emma Danko Prahbdip Singh
Adriano Paganini

Deadline: 12.03.2026

Inhaltsverzeichnis

1	Systemüberblick	2
1.1	Zielsetzung	3
1.2	Zielgruppe	3
1.3	Benutzer und Rollen	3
1.4	Kernfunktionalitäten	4
2	Use Case Diagramm	5
2.1	Akteure	5
2.2	Übersicht der Use Cases	6
3	Use Case Beschreibungen	6
3.1	UC-01: Raumklimadaten eigenes Büro einsehen	6
3.2	UC-02: Raumklimadaten Allgemeinflächen einsehen	7
3.3	UC-03: Abteilungsübersicht einsehen	7
3.4	UC-04: Grenzwertwarnung – Auslösung und Anzeige	8
3.5	UC-05: Grenzwerte konfigurieren	8
3.6	UC-06: Abwesenheit eintragen	9
3.7	UC-07: Geräte anlegen und konfigurieren	9
4	Klassendiagramm V1	10
4.1	Klassenbeschreibungen	10
4.2	Wichtige Assoziationen	12
4.3	Löschen von Daten	12
5	GUI Prototyp	13
5.1	Login	13
5.2	Dashboard (Mitarbeiter:in)	13
5.3	Verlaufsansicht (Raumdetails)	13
5.4	Dashboard (Abteilungsleitung)	15
5.5	Dashboard (Geschäftsführung)	15
5.6	Konfigurationsansicht (Hausverwalter)	15

5.7	Gerätemanagement (Systemadministrator)	16
6	Sequenzdiagramm: Normaler Messzyklus mit Grenzwertwarnung	17
7	Projektplan	18
7.1	Verantwortlichkeiten	18
7.2	Meilensteine und Zeitplan	19
7.3	Inkrementelle Entwicklung	19
7.4	Zusätzliche Aufgaben	20
8	Klassendiagramm V2	20
9	SW-Architektur	20
9.1	Laufzeitsicht	20
10	API-Dokumentation	20
11	Lösungsansatz komplexe Abläufe	22
12	Ausfallssicherheit	22
12.1	Szenario 1 – Ausfall der Sensorstation	22
12.2	Szenario 2 – Eingeschränkte BLE-Kommunikation	23
12.3	Szenario 3 – Unerwarteter Neustart des Raspberry Pi	24
12.4	Szenario 4 – Temporärer Ausfall: Raspberry Pi – Backend	25
12.5	Szenario 5 – Kurzfristiger Ausfall des zentralen Backends	26

1 Systemüberblick

ClimateCanary überwacht das Raumklima in Bürogebäuden über ein dreischichtiges IoT-System: Sensorstationen (Arduino) messen kontinuierlich und übertragen die Daten per BLE an einen Raspberry Pi pro Raum, der die Grenzwertlogik lokal ausführt und Daten datenschutzkonform an ein zentrales Spring Boot Backend weiterleitet. Nutzer sehen die Daten rollenabhängig in einer React-Webanwendung.

Wichtige Details:

- **BLE-Kopplung ist exklusiv:** Nach der ersten erfolgreichen BLE-Verbindung koppelt sich der Arduino ausschließlich mit *einem* Raspberry Pi – andere Geräte werden abgewiesen. Die Arduino-ID wird vom Systemadmin generiert und hardcoded auf dem Gerät hinterlegt.
- **Datenschutz gilt nur für Büroräume:** Die 5-Personen-Mindestbelegungsregel gilt ausschließlich für Büroräume (`RoomType.OFFICE`). Allgemeinflächen (`COMMON_AREA`) haben keinerlei Einschränkungen – dort wird immer gespeichert.
- **Transiente Verarbeitung ist immer erlaubt:** Auch wenn die Mindestbelegung unterschritten ist, darf der Raspberry Pi Messdaten für die Grenzwertberechnung kurzfristig im Speicher halten – er darf sie nur nicht persistent in SQLite speichern oder ans Backend übertragen.
- **Grenzwertverletzungen brauchen einen Noise-Filter:** Eine Warnung darf erst ausgelöst werden, wenn der Grenzwert über mehrere aufeinanderfolgende Messungen überschritten wird (konfigurierbar). Einzelne Ausreißer sollen keine Warnung erzeugen.
- **Raspberry Pi hat keinen eigenen REST-Server:** Der RPi betreibt nur einen REST-Client. Konfigurationsänderungen holt er sich selbst per Polling – die Webapp schickt keine aktiven Benachrichtigungen an den RPi.
- **Gebäudeadmin und Systemadmin sind strikt getrennt:** Der Hausverwalter sieht Raumklimadaten, aber *niemals* welche User welchem Raum zugeordnet sind. Der Systemadmin sieht Userdaten, aber *niemals* Messdaten. Diese Trennung ist kein Nice-to-have, sondern eine explizite Datenschutzanforderung.
- **Abteilungsleitung sieht nur Tagesdurchschnitte für Büros:** Die reduzierte Granularität für Büroräume ist eine bewusste Datenschutzentscheidung – Tagesdurchschnitte verhindern, dass aus Minutenwerten auf das Verhalten einzelner Personen geschlossen werden kann.
- **Raspberry Pi muss ausfallsicher sein:** Automatischer Neustart nach Stromausfall (Docker restart policy / systemd), Pufferung von Messdaten bei Backend-Ausfall mit Retry-Logik, und Reload der Konfiguration aus der lokalen SQLite-DB ohne Verbindung zum Backend.
- **Mehrere Teams senden BLE-Pakete:** Da andere Projektteams ebenfalls BLE-Geräte betreiben, muss die eigene Sensorstation in den Advertising-Paketen eindeutig als zum eigenen System gehörend identifizierbar sein.
- **SonarQube und Tests von Anfang an:** Mindestens 65% JUnit-Testabdeckung

ist eine Abgabebedingung – SonarQube soll nicht erst kurz vor der Deadline eingebunden werden.

1.1 Zielsetzung

- ClimateCanary ist ein IoT-basiertes Monitoring-System, das Unternehmen eine kontinuierliche und automatisierte Überwachung des Raumklimas in Bürogebäuden ermöglicht.
- Das System erfasst Temperatur, Luftfeuchtigkeit und Luftqualität in Echtzeit und stellt diese Daten rollenabhängig in einer zentralen Webanwendung dar.
- Kurzfristiges Ziel: Mitarbeiter:innen und Verwaltung sollen jederzeit informiert sein, wenn Raumklimawerte kritische Grenzwerte über- oder unterschreiten, und können entsprechend reagieren.
- Mittelfristiges Ziel: Langfristige Auswertungen der gesammelten Daten sollen als Entscheidungsgrundlage für gezielte Maßnahmen zur Raumklimaverbesserung dienen (z.B. Anschaffung von Klimaanlage, Luftreinigern).
- Langfristiges Ziel (außerhalb des aktuellen Projektumfangs): Automatische Steuerung von Heizung, Jalousien und Fenstern auf Basis der gesammelten Klimadaten.

1.2 Zielgruppe

- Primäre Zielgruppe sind mittelständische bis große Unternehmen mit mehreren Büroräumen und Allgemeinflächen.
- Das System richtet sich an alle Mitarbeiter:innen eines Unternehmens, die das Raumklima an ihrem Arbeitsplatz überwachen möchten.
- Zusätzlich adressiert es Abteilungsleitungen, das höhere Management sowie die Gebäude- und Systemadministration, die das System konfigurieren und übergreifend auswerten.

1.3 Benutzer und Rollen

- **Mitarbeiter:in:** Überwacht das Raumklima am eigenen Arbeitsplatz und in Allgemeinflächen der eigenen Abteilung. Verwaltet eigene Abwesenheiten.
- **Abteilungsleitung:** Erbt alle Rechte der Mitarbeiter:in. Erhält zusätzlich eine Übersicht aller Räume der eigenen Abteilung in reduzierter Datengranularität (Tagesdurchschnitte für Büroräume). Sieht Abwesenheiten der Mitarbeiter:innen.
- **Geschäftsführung:** Sieht firmenweite, aggregierte und anonymisierte Übersichten und Trendindikatoren. Kein Zugriff auf raumgenaue Daten.
- **Hausverwalter:** Sieht alle Raumklimadaten aller Räume, konfiguriert Grenzwerte und verwaltet Raumklima-Tipps. Kein Zugriff auf Userdaten oder Raumzuweisungen.
- **Systemadministrator:** Verwaltet Benutzer, Gebäudestruktur und Geräte (Raspberry Pis, Sensorstationen). Kein Zugriff auf Messdaten.

Wichtig: Hausverwalter und Systemadministrator sind strikt voneinander getrennt – diese Trennung ist das zentrale Datenschutzmechanismus des Systems. Der Hausverwalter sieht nie, welche User welchem Raum zugeordnet sind.

1.4 Kernfunktionalitäten

- Automatische Erfassung von Temperatur, Luftfeuchtigkeit und Luftqualität durch Arduino-Sensorstationen (BME680/688 Sensor).
- Übertragung der Messdaten via Bluetooth Low Energy (BLE) an einen raumgebundenen Raspberry Pi. Das BLE-Protokoll (GATT-Profil) wird selbst definiert; nach der ersten erfolgreichen Verbindung koppelt sich der Arduino ausschließlich mit dem zugewiesenen Raspberry Pi.
- Lokal am Raspberry Pi: Grenzwertprüfung mit Noise-Filter (Hysterese/Zeitfenster), datenschutzkonforme Zwischenspeicherung in SQLite, Weiterleitung an den zentralen Webserver per REST.
- Visuelle Rückmeldung direkt an der Sensorstation: LED-Ampel (Grün/Gelb/Rot) und LCD-Display mit aktuellen Werten, Warnungen und Tipps.
- Zentrale Webanwendung (Spring Boot + React/TypeScript) mit rollenabhängigen Ansichten, Verlaufsdiagrammen, Grenzwertwarnungen und Konfigurationsoptionen.
- Datenschutzkonforme Datenhaltung: Bürodaten werden nur bei Mindestbelegung von 5 Personen persistent gespeichert. Transiente Verarbeitung am Raspberry Pi (z.B. für Grenzwertberechnung) ist immer erlaubt.
- Abwesenheitssystem zur Pflege von Urlaub, Krankheit etc., das automatisch die Datenschutzprüfung beeinflusst und den zuständigen Raspberry Pi benachrichtigt.

2 Use Case Diagramm

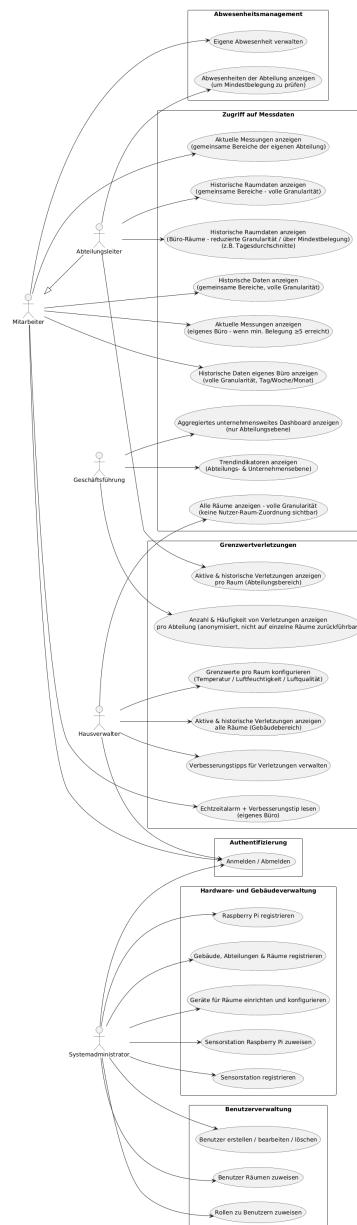


Abbildung 1: Use Case Diagramm

Das vollständige Use Case Diagramm befindet sich als separates PNG im `/images` Ordner (Datei: `images/Use_Case_Diagramm.png`).

2.1 Akteure

- **Mitarbeiter:in** – Basisrolle; hat Zugriff auf eigenes Büro (bei ausreichender Belegung) und Allgemeinflächen der eigenen Abteilung.
- **Abteilungsleitung** – Erbt alle Rechte der Mitarbeiter:in; zusätzlich Abteilungsübersicht mit reduzierter Granularität für Büroräume.
- **Geschäftsführung** – Nur firmenweite, aggregierte Sicht auf Abteilungsebene; keine Raumdetaillens, keine Userdaten.

- **Hausverwalter** – Vollzugriff auf Raumklimadaten und Konfiguration; sieht ausdrücklich *keine* Nutzer-Raum-Zuordnungen oder Abwesenheiten.
- **Systemadministrator** – Vollzugriff auf Benutzer- und Geräteverwaltung; kein Zugriff auf Messdaten oder Grenzwertkonfiguration.

2.2 Übersicht der Use Cases

Akteur	Use Cases
Mitarbeiter:in	Anmelden/Abmelden, aktuelle Messungen des eigenen Büros einsehen (nur bei Mindestbelegung ≥ 5), historische Daten des eigenen Büros einsehen, aktuelle und historische Messungen gemeinsamer Bereiche der eigenen Abteilung einsehen, Echtzeitalarme und Verbesserungstipps bei Grenzwertverletzungen lesen, eigene Abwesenheiten verwalten
Abteilungsleitung	Historische Raumdaten von Büro-Räumen der Abteilung einsehen (reduzierte Granularität, z. B. Tagesdurchschnitte), historische Daten gemeinsamer Bereiche einsehen, aktive und historische Grenzwertverletzungen pro Raum im Abteilungsbereich einsehen, Abwesenheiten der Abteilung einsehen
Geschäftsführung	Aggregiertes unternehmensweites Dashboard einsehen (auf Abteilungsebene), Trendindikatoren für Abteilungen und Unternehmen einsehen, anonymisierte Statistiken zu Grenzwertverletzungen pro Abteilung einsehen
Hausverwalter	Alle Raumdaten mit voller Granularität einsehen (ohne Nutzer-Raum-Zuordnung), aktive und historische Grenzwertverletzungen aller Räume einsehen, Grenzwerte für Temperatur, Luftfeuchtigkeit und Luftqualität konfigurieren, Verbesserungstipps für Grenzwertverletzungen verwalten
Systemadministrator	Gebäude, Abteilungen und Räume registrieren und verwalten, Raspberry Pis registrieren, Sensorstationen registrieren und Raspberry Pis zuweisen, Geräte für Räume konfigurieren, Benutzer erstellen/bearbeiten/löschen, Rollen zuweisen, Benutzer Räumen zuweisen

3 Use Case Beschreibungen

3.1 UC-01: Raumklimadaten eigenes Büro einsehen

- **Akteur:** Mitarbeiter:in
- **Vorbedingung:** Nutzer:in ist eingeloggt und einem Büroraum zugeordnet. Mindestens 5 Personen des Raums sind aktuell anwesend (keine Abwesenheit erfasst).
- **Normalablauf:**
 1. Nutzer:in öffnet die Raumansicht des eigenen Büros.
 2. System prüft die aktuelle Belegung des Raums anhand der eingetragenen Abwesenheiten.

3. System zeigt aktuelle Messwerte (Temperatur, Luftfeuchtigkeit, Luftqualität) und Verlaufsdiagramme (Tages-/Wochen-/Monatsansicht) an.
 4. Konfigurierte Grenzwerte sind als Referenzlinien in den Diagrammen sichtbar.
 5. Aktive Grenzwertwarnungen werden prominent angezeigt, inkl. Raumklima-Tipps.
- **Alternativablauf – Mindestbelegung nicht erfüllt:**
 1. System erkennt, dass die Belegung unter 5 Personen liegt.
 2. System zeigt Hinweis: „Keine Daten verfügbar – Datenschutzbedingungen nicht erfüllt.“
 3. Keine Messdaten werden angezeigt.
 - **Alternativablauf – Verbindungsausfall:**
 1. Für einen bestimmten Zeitraum fehlen Messdaten (z.B. Raspberry Pi offline).
 2. System markiert Lücken in den Verlaufsdiagrammen deutlich (z.B. grau hinterlegte Bereiche).

3.2 UC-02: Raumklimadaten Allgemeinflächen einsehen

- **Akteur:** Mitarbeiter:in, Abteilungsleitung
- **Vorbedingung:** Nutzer:in ist eingeloggt und einer Abteilung zugeordnet.
- **Normalablauf:**
 1. Nutzer:in wählt eine Allgemeinfläche (Konferenzraum, Aufenthaltsraum etc.) der eigenen Abteilung aus.
 2. System zeigt aktuelle Messwerte und Verlaufsdiagramme in voller Granularität an.
 3. Keine Datenschutzeinschränkungen – Daten sind immer verfügbar.

3.3 UC-03: Abteilungsübersicht einsehen

- **Akteur:** Abteilungsleitung
- **Vorbedingung:** Nutzer:in ist als Abteilungsleitung eingeloggt.
- **Normalablauf:**
 1. Abteilungsleitung öffnet die Abteilungsübersicht.
 2. System zeigt eine vergleichende Darstellung aller Räume der Abteilung (aktuelle Messwerte, Anzahl aktiver Warnungen).
 3. Für Büros oberhalb der Mindestbelegung: Verlaufsansichten in reduzierter Granularität (Tagesdurchschnitte).
 4. Für Allgemeinflächen: Verlaufsansichten in voller Granularität.
 5. Aktive und vergangene Grenzwertverletzungen auf Raumebene sind einsehbar.

3.4 UC-04: Grenzwertwarnung – Auslösung und Anzeige

- **Akteur:** Raspberry Pi (technisch), Mitarbeiter:in / Abteilungsleitung (sichtbar)
- **Vorbedingung:** Grenzwerte sind für den betroffenen Raum konfiguriert. Sensorstation ist aktiv und mit dem Raspberry Pi verbunden.
- **Normalablauf:**
 1. Raspberry Pi empfängt Messdaten von der Sensorstation via BLE.
 2. Raspberry Pi prüft Messwerte gegen konfigurierte Grenzwerte mit Noise-Filter (kein Auslösen bei kurzfristigen Ausreißern).
 3. Bei bestätigter Grenzwertüberschreitung: Raspberry Pi setzt Warnstatus und übermittelt diesen an die Sensorstation (via BLE) und die Webapp (via REST POST).
 4. Sensorstation: LED wechselt auf Rot, Display zeigt Warnmodus mit Hinweis und Tipp.
 5. Webapp: Warnung wird prominent für alle Nutzer:innen des betroffenen Raums angezeigt, inkl. konfiguriertem Raumklima-Tipp.
 6. Abteilungsleitung sieht Warnung in der Abteilungsübersicht.
- **Alternativablauf – Warnung aufheben:**
 1. Messwerte kehren dauerhaft in den Normalbereich zurück.
 2. Raspberry Pi hebt Warnstatus auf und informiert Sensorstation und Webapp.
 3. LED wechselt auf Grün, Warnmeldung in der Webapp verschwindet. Vergangene Verletzung bleibt historisch gespeichert (`violationStatus = RESOLVED`).
- **Designentscheidung:** Eine Grenzwertwarnung wird erst ausgelöst, wenn ein Grenzwert über einen konfigurierbaren Zeitraum (z.B. 3 aufeinanderfolgende Messungen) überschritten wird. Dadurch werden kurzfristige Ereignisse (Fenster kurz öffnen, Atemstoß auf Sensor) herausgefiltert. Die Anzahl der nötigen Messungen ist konfigurierbar und in der SQLite-Datenbank des Raspberry Pi hinterlegt.

3.5 UC-05: Grenzwerte konfigurieren

- **Akteur:** Hausverwalter
- **Vorbedingung:** Nutzer:in ist als Hausverwalter eingeloggt. Raum und Sensorstation sind im System angelegt.
- **Normalablauf:**
 1. Hausverwalter öffnet die Konfigurationsansicht eines Raums.
 2. Admin setzt oder ändert obere Grenzwerte für Temperatur, Luftfeuchtigkeit und Luftqualität. Untere Grenzwerte sind optional.
 3. System speichert die neuen Grenzwerte in der PostgreSQL-Datenbank.

4. Der zuständige Raspberry Pi holt sich die aktualisierten Grenzwerte beim nächsten regulären Polling-Intervall selbstständig per GET-Request ab.
 5. Raspberry Pi speichert die neuen Grenzwerte lokal in der SQLite-Datenbank und wendet sie ab der nächsten Messung an.
- **Designentscheidung:** Der Raspberry Pi fragt Konfigurationsänderungen aktiv per Polling ab – er betreibt keinen eigenen REST-Server. Das vereinfacht die Implementierung am Edge-Device. Der Nachteil (leichte Verzögerung bei Konfigurationsänderungen) ist für einen Prototyp akzeptabel.

3.6 UC-06: Abwesenheit eintragen

- **Akteur:** Mitarbeiter:in
- **Vorbedingung:** Nutzer:in ist eingeloggt und einem Büroraum zugeordnet.
- **Normalablauf:**
 1. Mitarbeiter:in trägt eine Abwesenheit (Urlaub, Krankheit, etc.) stunden- oder tageweise ein.
 2. System prüft, ob durch diese Abwesenheit die Mindestbelegung (5 Personen) im Büroraum unterschritten wird.
 3. Wenn ja: Das Backend setzt ein Flag im Raum-Datensatz (`belowMinOccupancy = true`). Der Raspberry Pi erkennt dies beim nächsten Polling und speichert ab diesem Zeitpunkt keine Messdaten mehr persistent.
 4. Wenn nein: Keine Änderung am Datenspeicherverhalten.

3.7 UC-07: Geräte anlegen und konfigurieren

- **Akteur:** Systemadministrator
- **Vorbedingung:** Nutzer:in ist als Systemadministrator eingeloggt. Gebäudestruktur (Gebäude, Abteilungen, Räume) ist bereits angelegt.
- **Normalablauf:**
 1. Systemadministrator legt ein neues Raspberry Pi-Objekt an. System generiert automatisch eine eindeutige ID.
 2. Systemadministrator ordnet den Raspberry Pi einem Raum zu.
 3. Systemadministrator legt eine neue Sensorstation an. System generiert automatisch eine eindeutige ID.
 4. Systemadministrator ordnet die Sensorstation dem Raspberry Pi zu.
 5. Die generierten IDs werden für die physische Konfiguration der Geräte verwendet (hardcoded auf Arduino, `conf.yaml` auf Raspberry Pi).

4 Klassendiagramm V1

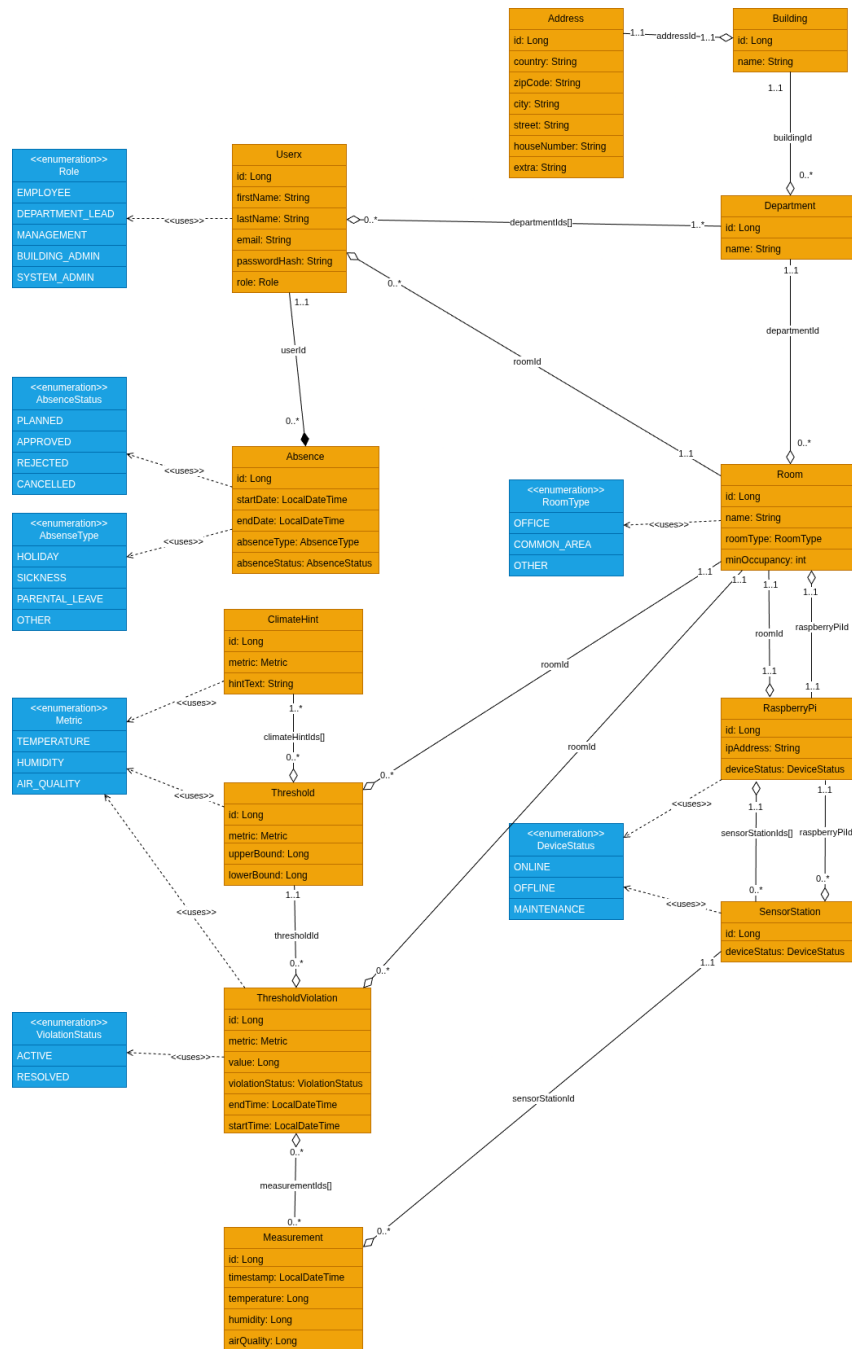


Abbildung 2: Klassendiagramm V1 – fachliches Modell der Webapp-Backend-Daten

Das vollständige Klassendiagramm befindet sich als Draw.io Datei im GitLab Wiki (Datei: *images/climate-canary-webapp-backend.png*).

4.1 Klassenbeschreibungen

- **ViolationStatus** – Enum: ACTIVE, RESOLVED.
- **Role** – Enum: EMPLOYEE, DEPARTMENT_LEAD, MANAGEMENT, BUILDING_ADMIN, SYSTEM_ADMIN.

- **AbsenceType** – Enum: HOLIDAY, SICKNESS, PARENTAL_LEAVE, OTHER.
- **AbsenceStatus** – Enum: PLANNED, APPROVED, REJECTED, CANCELLED.
- **RoomType** – Enum: OFFICE, COMMON_AREA, OTHER.
- **DeviceStatus** – Enum: ONLINE, OFFLINE, MAINTENANCE.
- **Metric** – Enum: TEMPERATURE, HUMIDITY, AIR_QUALITY.
- **User** – Repräsentiert einen Systembenutzer. Attribute: id, firstName, lastName, email, passwordHash, role, departmentIds, roomId. Assoziationen: gehört einer oder mehreren Abteilungen an, ist einem Büroraum zugeordnet, hat Abwesenheiten. *Designentscheidung: Ein User kann mehreren Abteilungen angehören, um z.B. Management- oder Gebäudeadministrationsrollen zu ermöglichen, die Zugriff auf mehrere Abteilungen benötigen.*
- **Absence** – Repräsentiert eine Abwesenheit eines Users. Attribute: id, userId, startDate, endDate, absenceType, absenceStatus. Assoziationen: referenziert einen User.
- **Building** – Repräsentiert ein Gebäude. Attribute: id, name, addressId. Assoziationen: hat eine Adresse, wird von mehreren Abteilungen referenziert.
- **Address** – Repräsentiert eine Adresse. Attribute: id, country, zipCode, city, street, houseNumber, extra.
- **Department** – Repräsentiert eine Abteilung. Attribute: id, name, buildingId. Assoziationen: gehört zu einem Gebäude, wird von mehreren Räumen und Usern referenziert.
- **Room** – Repräsentiert einen Raum. Attribute: id, name, roomType, departmentId, minOccupancy (default: 5 für Büroräume), raspberryPiId. Assoziationen: hat einen Raspberry Pi, wird von Grenzwerten und Grenzwertüberschreitungen referenziert.
- **RaspberryPi** – Repräsentiert einen Raspberry Pi. Attribute: id (generiert), roomId, ipAddress, deviceStatus, sensorStationIds. Assoziationen: hat Sensorstationen; Grenzwerte werden über die Raumzuweisung verwaltet.
- **SensorStation** – Repräsentiert eine Sensorstation (Arduino). Attribute: id (generiert, hardcoded auf Arduino), raspberryPiId, deviceStatus. Assoziationen: gehört zu einem Raspberry Pi, wird von Messungen referenziert.
- **Measurement** – Repräsentiert einen einzelnen Messdatenpunkt. Attribute: id, sensorStationId, timestamp, temperature, humidity, airQuality. Wird *nur* bei erfüllter Datenschutzbedingung (Mindestbelegung) persistent gespeichert.
- **Threshold** – Repräsentiert Grenzwerte für einen Raum pro Messgröße. Attribute: id, roomId, metric, upperBound, lowerBound (optional). *Designentscheidung: Ein oberer Grenzwert ist immer verpflichtend; der untere Grenzwert ist optional und muss im Konzept begründet werden, wenn er weggelassen wird.*
- **ThresholdViolation** – Repräsentiert eine aktive oder historische Grenzwertverletzung. Attribute: id, roomId, thresholdId, startTime, endTime (null solange aktiv), metric, value, violationStatus, measurementIds. *Designentscheidung: Mehrere*

Measurements müssen herangezogen werden, um kurze Extremwerte zu filtern. Eine Verletzung wird erst bestätigt, wenn z.B. 3 aufeinanderfolgende Messungen den Grenzwert überschreiten.

- **ClimateHint** – Repräsentiert einen Raumklima-Tipp. Attribute: id, metric, hint-Text. Wird bei Grenzwertverletzungen auf dem LCD-Display der Sensorstation und in der Webapp angezeigt. Assoziationen: wird von einem oder mehreren Thresholds referenziert.

4.2 Wichtige Assoziationen

- **Building 1 – * Department**: Ein Gebäude hat mehrere Abteilungen.
- **Department 1 – * Room**: Eine Abteilung hat mehrere Räume.
- **Department 1 – * User**: Eine Abteilung hat mehrere User.
- **Room 1 – 0..1 RaspberryPi**: Ein Raum hat maximal einen Raspberry Pi.
- **RaspberryPi 1 – * SensorStation**: Ein Raspberry Pi kann mehrere Sensorstationen verwalten (im Prototyp: eine pro Raum).
- **SensorStation 1 – * Measurement**: Eine Sensorstation produziert viele Messdaten.
- **Room 1 – * Threshold**: Ein Raum hat Grenzwerte pro Messgröße (max. 3: Temperatur, Luftfeuchtigkeit, Luftqualität).
- **User 1 – * Absence**: Ein User kann mehrere Abwesenheiten haben.

4.3 Löschen von Daten

- Wird ein User gelöscht, werden seine Abwesenheiten ebenfalls gelöscht (CASCADE).
- Wird eine Sensorstation gelöscht, werden ihre Messdaten ebenfalls gelöscht (CASCADE). Historische Grenzwertverletzungen bleiben erhalten (roomId-Referenz bleibt bestehen).
- Wird ein Raum gelöscht, werden alle zugehörigen Messdaten, Grenzwerte und Grenzwertverletzungen gelöscht (CASCADE).
- Wird ein Raspberry Pi aus einem Raum entfernt, bleiben historische Messdaten erhalten; der Raspberry Pi-Eintrag bleibt im System, bis er explizit gelöscht wird.
- Messdaten, die aufgrund der Datenschutzbedingung (Mindestbelegung) nicht gespeichert wurden, existieren nie persistent – sie müssen also nicht gelöscht werden.

5 GUI Prototyp

Die vollständigen Papierprototypen der GUI befinden sich im Repository unter *docs/Softwarekonzept_V1/images/gui_prototype*. Folgend werden die Kernansichten textuell beschrieben, weiters werden einzelne Prototypen direkt im Dokument eingebunden.

5.1 Login

- Einfache Login-Maske mit E-Mail und Passwort.
- Nach Login: Weiterleitung auf rollenabhängiges Dashboard.



Abbildung 3: Papierprototyp der Login-Ansicht

5.2 Dashboard (Mitarbeiter:in)

- Übersichtskacheln: Eigenes Büro (aktuelle Temperatur, Luftfeuchtigkeit, Luftqualität) + Allgemeinflächen der Abteilung.
- Prominente Warnanzeige wenn aktive Grenzwertverletzung im eigenen Büro vorhanden, inkl. zugehörigem Raumklima-Tipp.
- Navigation zu: Verlaufsansicht, Abwesenheitsverwaltung.

5.3 Verlaufsansicht (Raumdetails)

- Liniendiagramm für Temperatur, Luftfeuchtigkeit, Luftqualität über wählbaren Zeitraum (Tag / Woche / Monat).
- Grenzwerte als horizontale Referenzlinien eingeblendet.
- Vergangene Grenzwertverletzungen farblich markiert.

① MA-Dashboard

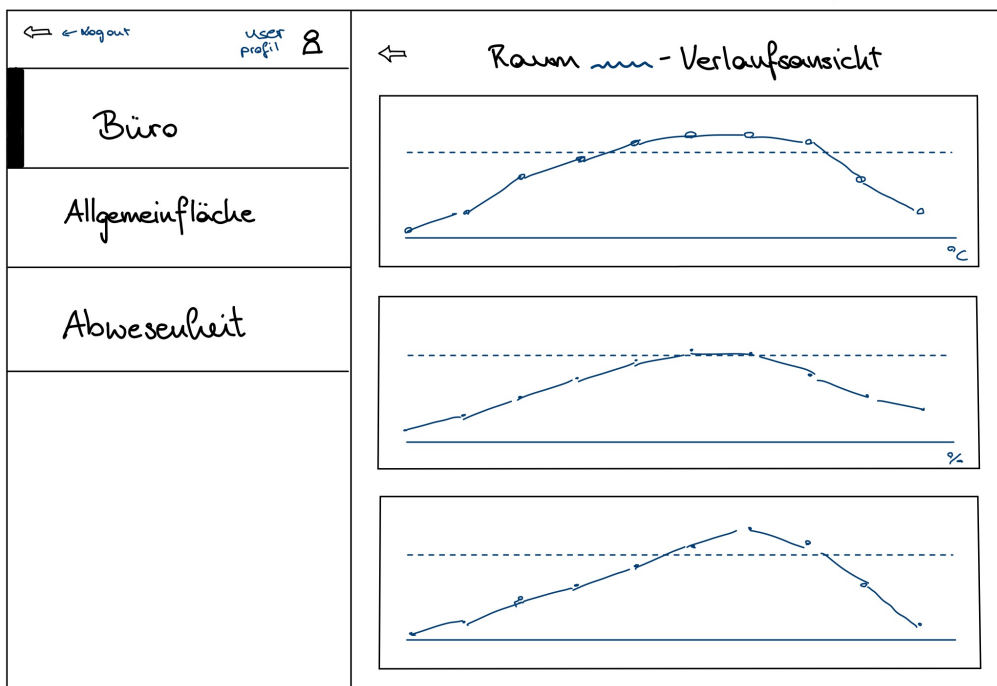
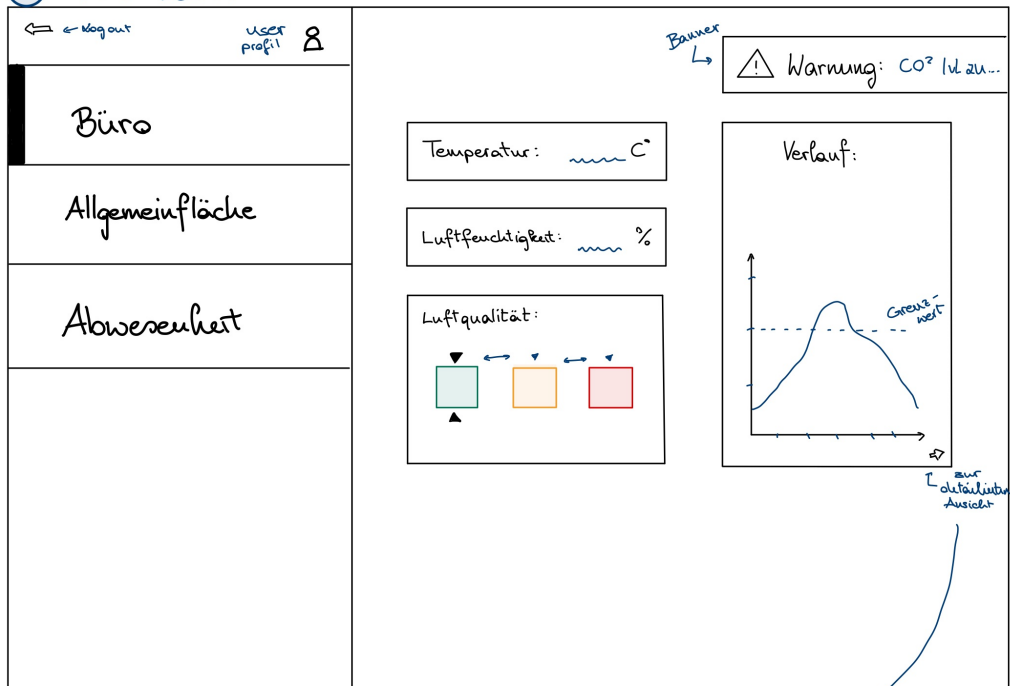


Abbildung 4: Papierprototyp des Dashboards für Mitarbeiter:innen mit Verlaufsansicht und Abwesenheitsverwaltung

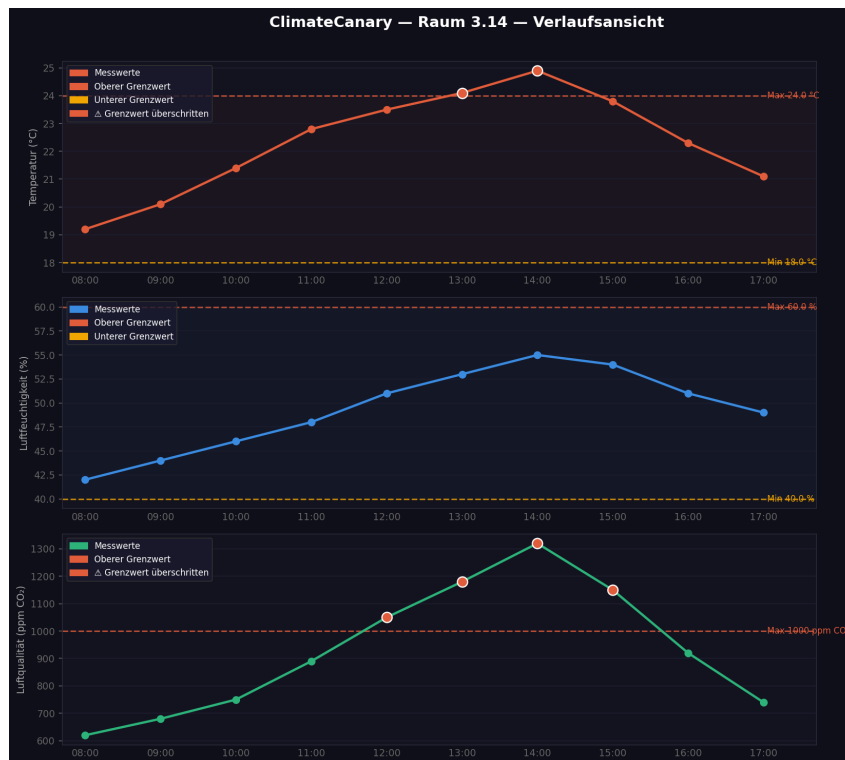


Abbildung 5: GUI Prototyp – Verlaufsansicht mit Grenzwert-Referenzlinien

- Datenlücken (Datenschutz, Ausfall) klar als graue Bereiche gekennzeichnet.
- Granularität abhängig von Rolle: Mitarbeiter:in sieht Rohdaten; Abteilungsleitung sieht Tagesdurchschnitte für Büroräume.

5.4 Dashboard (Abteilungsleitung)

- Rasteransicht aller Räume der Abteilung mit aktuellen Messwerten und Warnstatus.
- Anzahl aktiver Grenzwertverletzungen pro Raum.
- Verlinkung zu Raumdetailansicht mit reduzierter Granularität für Büros.

5.5 Dashboard (Geschäftsführung)

- Firmenweites Dashboard: Aggregierte Werte nach Abteilungen – keine raumgenauen Daten.
- Balkendiagramm: Anzahl Grenzwertverletzungen pro Abteilung (anonymisiert, nicht auf einzelne Räume zurückführbar).
- Trendlinie über Wochen.

5.6 Konfigurationsansicht (Hausverwalter)

- Liste aller Räume mit aktuellen Grenzwerten.
- Inline-Bearbeitung der Grenzwerte pro Raum und Messgröße.

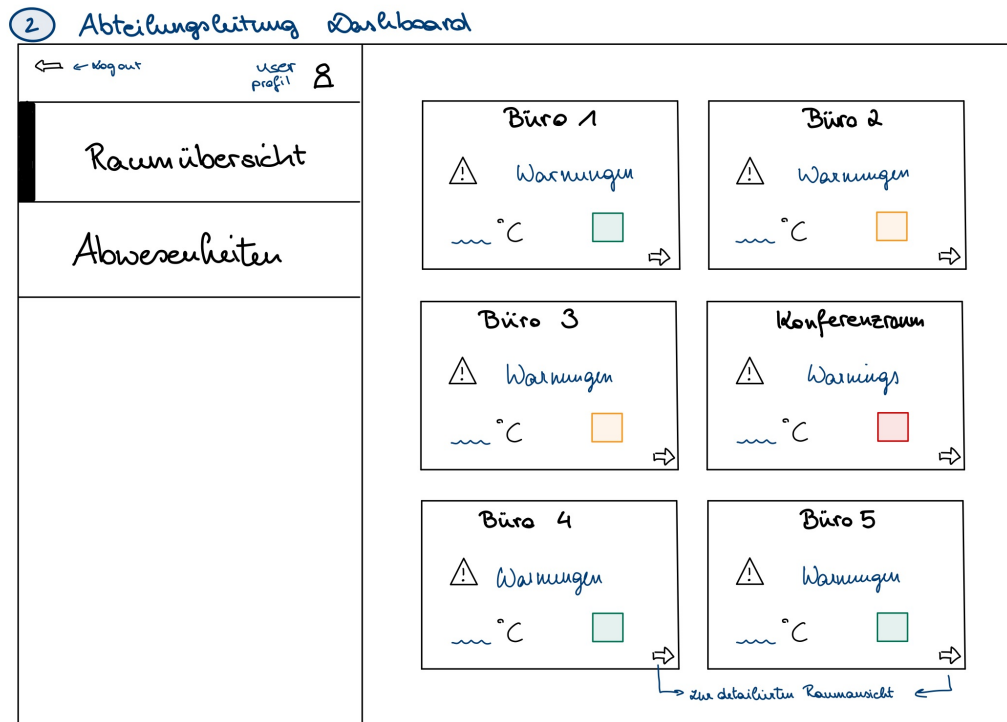


Abbildung 6: Papierprototyp des Dashboards der Abteilungsleitung mit Raumübersicht

- Tipp-Verwaltung: Liste, Erstellen, Bearbeiten, Löschen von Raumklima-Tipps.

5.7 Gerätemanagement (Systemadministrator)

- Übersicht aller Raspberry Pis und Sensorstationen mit Status (Online/Offline/Maintenance).
- Anlegen neuer Geräte inkl. automatisch generierter ID.
- Zuweisung von Sensorstationen zu Raspberry Pis und Räumen.
- Benutzerverwaltung: Nutzer:innen anlegen, Rollen zuweisen, Räumen zuordnen.

6 Sequenzdiagramm: Normaler Messzyklus mit Grenzwertwarnung

Das vollständige Sequenzdiagramm befindet sich als separates PNG im /images Ordner (Datei: *images/general_sequence_diagram.png*).

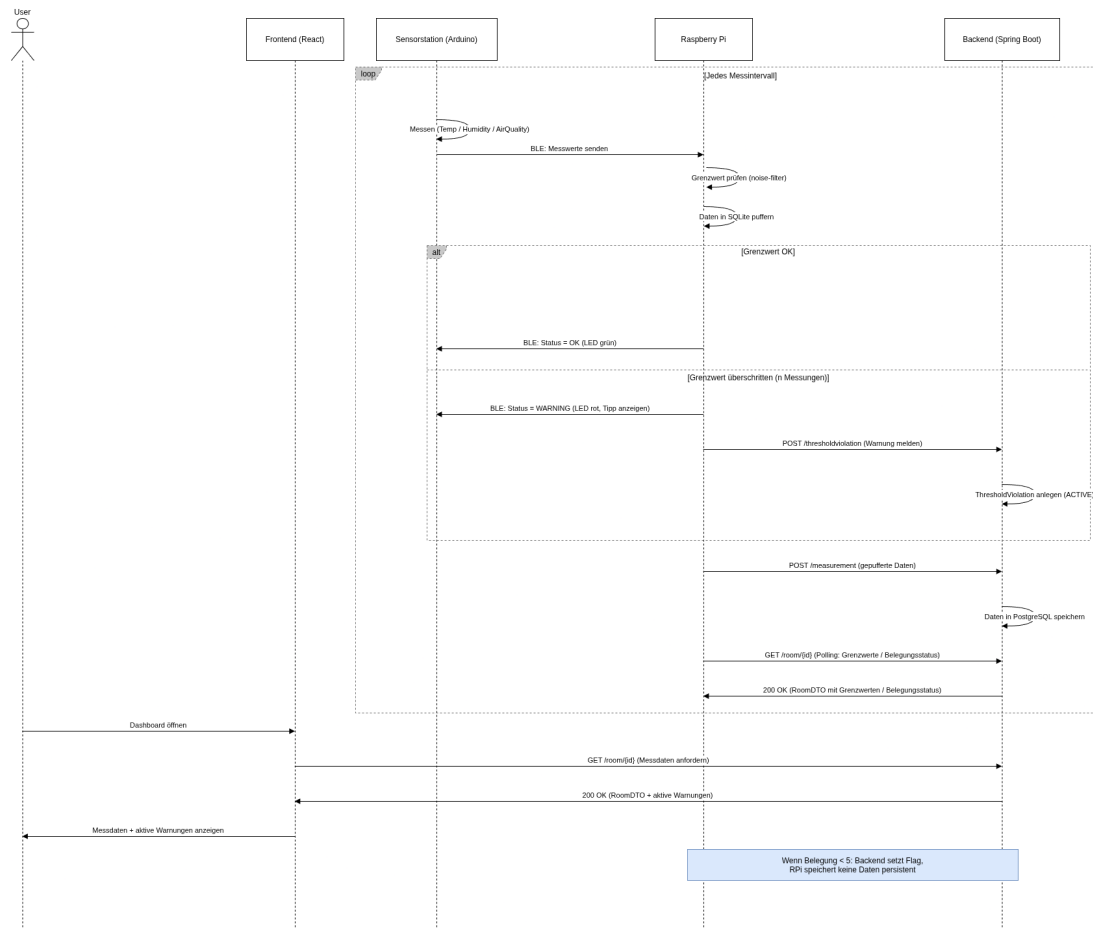


Abbildung 7: Erster Entwurf eines Sequenzdiagramms des allgemeinen Ablaufs des gesamten Systems

Dieser Sequenzdiagramm Entwurf zeigt den typischen Ablauf eines Messzyklus – von der Messung am Arduino bis zur Anzeige in der Webapp – inklusive dem Auslösen und Aufheben einer Grenzwertwarnung.

Hinweis: Das Sequenzdiagramm wird in V2 um weitere Abläufe ergänzt (Verbindungs Aufbau, Datenschutz-Abschaltung, Konfigurationsänderung).

7 Projektplan

7.1 Verantwortlichkeiten

Person	Zuständigkeit
Maria Kuhn	Arduino-Entwicklung (C/C++, Display/LED/Buttons), Frontend (React/TS)
Fabienne Schedler	Frontend (React/TS, PrimeReact, GUI-Prototyp)
Emma Danko	Raspberry Pi (Python, REST-Schnittstellen, BLE Central, SQLite, Docker)
Prahbdip Singh	Raspberry Pi (Python, REST-Schnittstellen, BLE Central, SQLite, Docker), Backend
Adriano Paganini	Arduino-Entwicklung (C/C++, Display/LED/Buttons), Backend
Alle	Softwarekonzept, Testing, Dokumentation, Code Review

7.2 Meilensteine und Zeitplan

Datum	Meilenstein	Inhalt
12.03.2026	Konzept V1	Systemüberblick, Use Cases, Klassendiagramm V1, GUI Prototyp, Projektplan
19.03.2026	Konzept V2	Sequenzdiagramme, API Dokumentation, Ausfallssicherheit, Klassendiagramm V2
27.03.2026	Konzept Final	Überarbeitetes Gesamtkonzept nach Feedback
KW 14–15	M1: Hardware Setup	Arduino + Sensoren funktionsfähig; BLE-Übertragung zum RPi; Raspberry Pi OS + Docker eingerichtet
KW 16–17	M2: Backend Grundgerüst	Spring Boot Skeleton aufgesetzt; PostgreSQL Datenbankschema implementiert; Basis-REST-API (OpenAPI); Login/Rollen
KW 18–19	M3: Datenfluss End-to-End	Kompletter Datenfluss Arduino → RPi → Backend funktionsfähig; Grenzwertberechnung am RPi; Datenschutzlogik
KW 20–21	M4: Frontend + Warnungen	Rollenabhängige Dashboards; Verlaufsdiagramme; Grenzwertwarnungen in Webapp und auf Sensorstation
KW 22–23	M5: Ausfallssicherheit + Testing	Ausfallszenarien abgesichert; SonarQube integriert; $\geq 65\%$ Testabdeckung
KW 24	M6: Finalisierung	Bugfixes; Dokumentation vollständig; finale Abgabe (18.06.2026)

7.3 Inkrementelle Entwicklung

- Wir entwickeln das System in zwei parallelen Tracks: **Hardware-Track** (Arduino + Raspberry Pi) und **Software-Track** (Backend + Frontend).
- Beide Tracks werden ab M3 integriert.
- Jede Woche wird im Team eine kurze Sync-Session abgehalten, um Fortschritte und Blocker zu besprechen.
- GitLab wird für Issues, Merge Requests und Code Review genutzt. Das Wiki enthält alle Konzept-Dokumente.
- SonarQube wird ab Beginn der Backend-Entwicklung (KW 14) integriert, nicht erst kurz vor der Abgabe.

7.4 Zusätzliche Aufgaben

- Zwischenbericht: Einplanen in KW 18 (nach M3).
- Abschlussbericht: Einplanen in KW 23–24.
- Präsentation: Vorbereitung in KW 24.
- Testdaten: Realistische Testdaten für alle Rollen und Szenarien werden in KW 20 erstellt.

8 Klassendiagramm V2

9 SW-Architektur

9.1 Laufzeitsicht

- Sequenzdiagramm - Grenzwertverletzung des gesamten Systems
- Sequenzdiagramm - Belegungs-Mindestgrenze (erledigt, siehe images/Sequenzdiagramm - Datenschutz Mindestbelegung)
- Sequenzdiagramm - Verbindungsaufbau und -konfiguration Arduino / Raspberry / Webapp

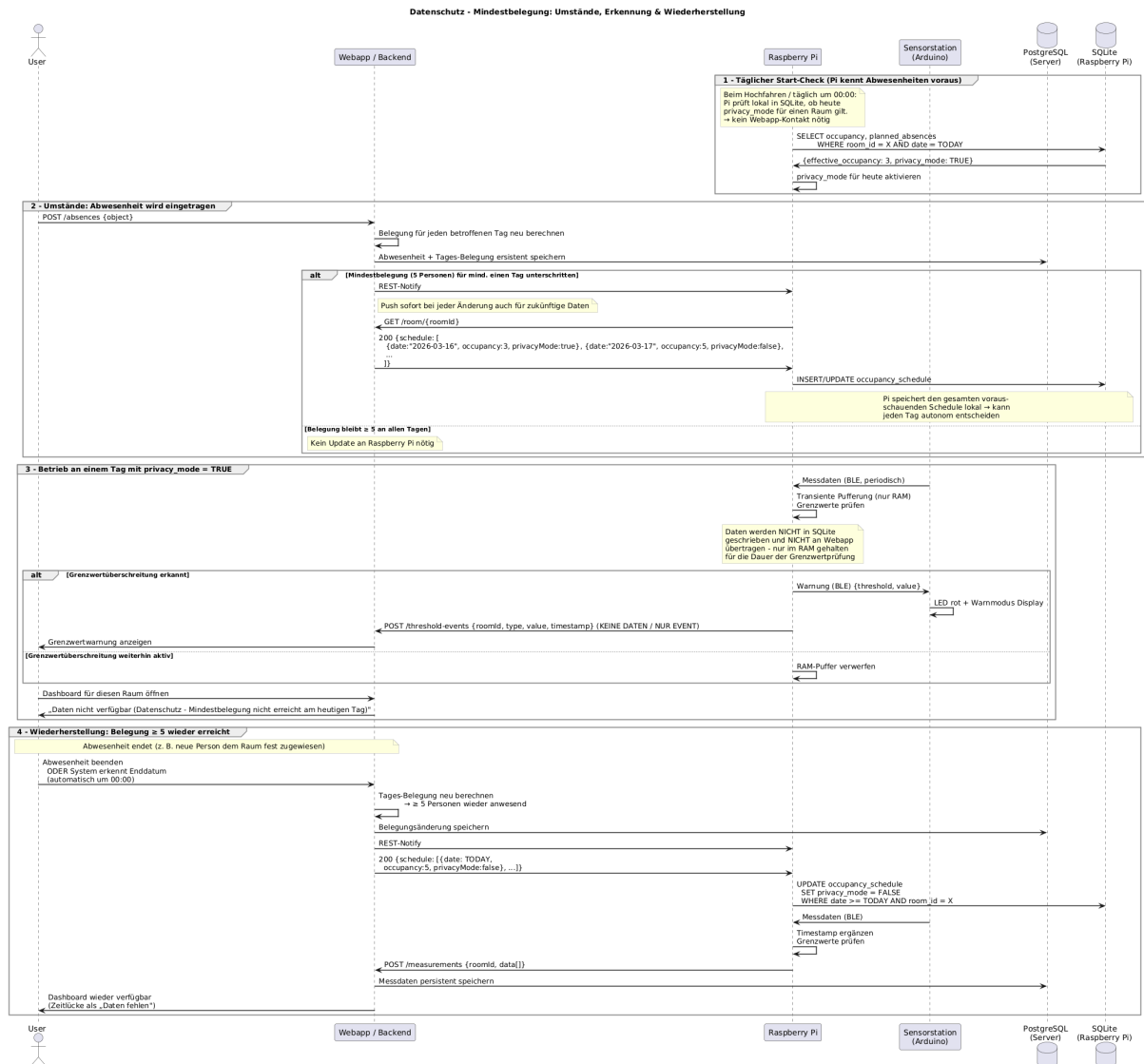


Abbildung 8: Sequenzdiagramm – Belegungs-Mindestgrenze

10 API-Dokumentation

siehe Wiki

11 Lösungsansatz komplexe Abläufe

12 Ausfallssicherheit

12.1 Szenario 1 – Ausfall der Sensorstation

Datenkonsistenz	Der Raspberry Pi erkennt den Ausfall über einen BLE-Verbindungs Timeout. Da keine neuen Messdaten eintreffen, werden keine fehlerhaften oder unvollständigen Einträge in die SQLite-Datenbank geschrieben. Bereits gepufferte Daten bleiben erhalten.
Logging	Der Raspberry Pi schreibt einen ERROR -Eintrag in sein lokales Logfile (sensor_offline , Sensor-ID, Timestamp). Der Webserver erhält ein entsprechendes Event und loggt den Ausfall ebenfalls auf ERROR -Level in beiden Server-Logfiles. Die Wiederverbindung wird auf INFO -Level protokolliert.
Wiederaufnahme	Sobald die Sensorstation wieder BLE-Advertising mit ihrer bekannten ID sendet, baut der Raspberry Pi automatisch die Verbindung wieder auf; kein manuelles Eingreifen ist notwendig.
Informationsaustausch	Der Raspberry Pi sendet bei Erkennung des Ausfalls aktiv ein POST /events {type: sensor_offline, sensorId} an die Webapp. Bei Wiederherstellung folgt ein entsprechendes sensor_online -Event.
User-Information	• Mitarbeiter:innen / Abteilungsleitung: Hinweis im Dashboard, dass die Sensorstation offline ist; Zeitlücke im Verlaufsdiagramm als „Daten nicht verfügbarmmarkiert.“ • Systemadmins: Gerätestatus in der Verwaltungsansicht zeigt Ausfall; in beiden Logfiles einsehbar. • Gebäudeadmins: Keine direkten Geräteinformationen (Datentrennung), aber fehlende Messdaten im Raumverlauf erkennbar.

12.2 Szenario 2 – Eingeschränkte BLE-Kommunikation

Datenkonsistenz	Die Sensorstation puffert Messwerte lokal im Arduino-Speicher, solange die BLE-Verbindung instabil ist. Der Raspberry Pi verwirft unvollständige Pakete und wartet auf eine stabile Übertragung. Nur vollständig empfangene und validierte Datenpakete werden in SQLite geschrieben.
Logging	Verbindungsprobleme werden auf WARN -Level im Raspberry-Pi-Logfile festgehalten, inkl. Anzahl verworfener Pakete und Dauer der Instabilität. Sobald die Verbindung wiederhergestellt ist, wird dies auf INFO -Level geloggt.
Wiederaufnahme	Nach Stabilisierung der BLE-Verbindung überträgt die Sensorstation die gepufferten Messwerte kumuliert. Der Raspberry Pi ergänzt Timestamps für jeden Messpunkt und verarbeitet die Daten regulär weiter.
Informationsaustausch	Während der Instabilität werden keine unvollständigen Daten an die Webapp weitergeleitet. Nach Wiederherstellung erfolgt die Übertragung der aufgelaufenen Daten über ein API Endpoint.
User-Information	• Mitarbeiter:innen / Abteilungsleitung: Lücke im Verlaufsdiagramm als „Daten fehlen (Verbindungsproblem)“ markiert; keine Echtzeit-Warnung. • Systemadmins: Verbindungsstatus der BLE-Verbindung in der Geräteverwaltung; WARN-Einträge in den Logfiles.

12.3 Szenario 3 – Unerwarteter Neustart des Raspberry Pi

Datenkonsistenz	Alle Datenbankoperationen auf der SQLite-Datenbank werden als ACID-konforme Transaktionen ausgeführt. Nach dem Neustart wird die Datenbank auf Integrität geprüft, bevor der Betrieb aufgenommen wird.
Logging	Der Neustart wird beim ersten Start des Dienstes als WARN -Eintrag protokolliert (erkennbar am Fehlen eines regulären shutdown -Eintrags). Der Webserver loggt den Reconnect des Raspberry Pi auf INFO -Level.
Wiederaufnahme	Der Raspberry-Pi-Dienst startet nach einem Neustart automatisch. Beim Start: 1. Laden der Konfiguration aus Config-File. 2. Abfrage der aktuellen Konfiguration anhand einer API End-point. 3. Automatischer Aufbau der BLE-Verbindungen zu zugeordneten Sensorstationen.
Informationsaustausch	Nach dem Reconnect sendet der Pi einen initialen Config-Request an den Webserver. Der Webserver erkennt den Reconnect und übermittelt ggf. ausstehende Konfigurationsänderungen.
User-Information	• Systemadmins / Gebäudeadmins: Kurzer Ausfall in den Logs und im Gerätestatus sichtbar. • Alle Rollen: Datenlücke im Dashboard entsprechend markiert.

12.4 Szenario 4 – Temporärer Ausfall: Raspberry Pi – Backend

Datenkonsistenz	Der Raspberry Pi puffert alle eingehenden Messdaten mit Time-stamps in SQLite. Doppelte Übertragungen werden durch eindeutige Measurement-IDs auf Serverseite verhindert.
Logging	Fehlgeschlagene Übertragungsversuche werden auf ERROR -Level geloggt. Die Wiederherstellung der Verbindung wird auf INFO -Level festgehalten.
Wiederaufnahme	Der Raspberry Pi implementiert eine Retry-Queue. Grenzwertberechnungen und die Kommunikation mit den Sensorstationen laufen während des Ausfalls vollständig autonom weiter; Grenzwerte liegen lokal in SQLite vor, sodass keine Webapp-Abhängigkeit besteht. Nach Wiederherstellung überträgt der Pi gepufferte Daten chronologisch geordnet.
Informationsaustausch	Nach Wiederherstellung der Verbindung überträgt der Pi die gepufferten Daten geordnet an den Webserver. Der Server markiert eventuelle Zeitlücken im Datensatz.
User-Information	• Mitarbeiter:innen / Abteilungsleitung: Hinweis auf fehlende Daten im betroffenen Zeitraum im Dashboard. • Systemadmins: Verbindungsstatus, Dauer des Ausfalls und Retry-Versuche in den Logfiles einsehbar. • Lokal (Sensorstation): LED und Display funktionieren weiterhin normal; Grenzwertwarnungen werden lokal angezeigt.

12.5 Szenario 5 – Kurzfristiger Ausfall des zentralen Backends

Datenkonsistenz	Da alle persistenten Messdaten primär über den Webserver in PostgreSQL gespeichert werden, entstehen bei einem Backend-Ausfall keine Schreibkonflikte. Der Raspberry Pi puffert in SQLite und überträgt nach Wiederherstellung. Der Webserver stellt beim Start die Datenbankverbindung zu PostgreSQL wieder her und verarbeitet aufgelaufene Requests aus der Pi-Retry-Queue.
Logging	Der Webserver schreibt beim Hochfahren einen INFO-Eintrag. Ein externer Health-Check detektiert den Ausfall und schreibt ihn in die Logfiles. Alle beim Reconnect eingehenden Verbindungsversuche der Raspberry Pis werden ebenfalls geloggt.
Wiederaufnahme	Nach dem Neustart authentifizieren sich die Raspberry Pis automatisch erneut und senden ihre gepufferten Daten. Die Webapp ist für alle Nutzer wieder erreichbar.
Informationsaustausch	Während des Ausfalls laufen Sensorstation und Raspberry Pi vollständig autonom weiter; Messung, Grenzwertprüfung und lokale Warnung über LED/Display funktionieren unabhängig vom Backend. Grenzwert-Events werden in der Pi-Retry-Queue zwischengespeichert und nach Wiederherstellung übertragen.
User-Information	• Mitarbeiter:innen / alle Webapp-Nutzer: Generische Fehlermeldung beim Öffnen der Webapp (HTTP 503 Service Unavailable). • Lokal (Sensorstation): Warnungen über Displays und LEDs funktionieren weiterhin uneingeschränkt. • Systemadmins: Ausfall-Zeitraum nach Wiederherstellung in den Logfiles einsehbar; Docker-Health-Check-Status protokolliert. • Alle Rollen: Datenlücke im Verlaufsdiagramm als „Daten fehlen“ gekennzeichnet.
